

Smart Chef

Code Documentation

*A detailed walkthrough of every file created for the Smart Chef application:
backend (FastAPI + SQLite), frontend (React), and the ML training scripts.*

Overview

Smart Chef is a full-stack application split into three parts: a Python/FastAPI backend that owns the database, authentication, and business logic; a React frontend that provides the mobile-first user interface; and a pair of standalone Python training scripts that produce the ingredient-recognition model consumed by the backend. This document goes through every source file that was written for the project, organized the same way the codebase itself is organized, explaining what each file does and why it was built that way.

The backend uses FastAPI with SQLAlchemy over a single SQLite database file, JWT-based authentication with bcrypt password hashing, TensorFlow/Keras for the image-classification model, and the OpenAI API for recipe generation. The frontend is a Vite-built React app using React Router for navigation and plain CSS (no UI framework) for a mobile-first, phone-like layout. There is no separate state-management library - each page manages its own data with React's built-in hooks and talks to the backend through a small shared fetch wrapper.

1. Backend

The backend lives under `backend/app/` and is organized into a handful of clear responsibilities: configuration and database setup, the data model, authentication, one router per resource, the ML classifier, and the OpenAI integration.

1.1 Configuration & Database Setup

backend/app/config.py

Role: Centralized application settings

This module defines the `Settings` class using `pydantic-settings`, which automatically reads configuration values from environment variables and the `backend/.env` file. It exposes three groups of settings: the JWT secret and algorithm used to sign login tokens, the token expiry window (one week), and the OpenAI API key and model name used for recipe generation. It also defines shared filesystem paths - `APP_DIR`, `MODEL_DIR` (where the trained classifier files live), and `DATABASE_PATH` (the SQLite file location) - so every other module references the same paths instead of hardcoding them. A single settings instance is created at import time and reused everywhere via `from app.config import settings`.

backend/app/database.py

Role: SQLAlchemy engine and session management

Sets up the connection to the single SQLite database file that stores the whole app's data. It creates the SQLAlchemy engine pointed at `DATABASE_PATH`, with `check_same_thread=False` so SQLite can be safely used across FastAPI's request-handling threads. `SessionLocal` is a session factory, and `Base` is the declarative base class every ORM model (in `models.py`) inherits from. The `get_db()` function is a FastAPI dependency: it opens a new database session per request and guarantees it's closed afterward via `try/finally` - the standard SQLAlchemy + FastAPI pattern for per-request sessions.

1.2 Data Models & Schemas

backend/app/models.py

Role: Database schema - the four tables

Defines the four SQLAlchemy ORM tables that make up the entire data model: `User`, `UserPreference`, `Ingredient`, and `Recipe`. `User` stores login credentials (hashed password, never plaintext) plus relationships to the other three tables, all with `cascade="all, delete-orphan"` so deleting an account automatically removes every ingredient, recipe, and preference the user owned. `UserPreference` is a one-to-one table holding the answers to the onboarding questionnaire - vegetarian/vegan flags, allergies and disliked foods (stored as JSON lists), diet info, and preferred difficulty/prep time - editable later from `Settings`. `Ingredient` rows represent what the user currently has at home (name, quantity, unit). `Recipe` stores every AI-generated recipe: its meal type, title, a JSON content blob holding the full recipe body (prep time, difficulty, ingredients used, steps), and an `is_favorite` flag. A single `Recipe` table intentionally serves both the `History` tab (every row) and the `Recipes/favorites` tab (filtered by `is_favorite`), avoiding a separate history table.

backend/app/schemas.py

Role: Pydantic request/response contracts

Defines the shape of every JSON payload the API accepts and returns, using Pydantic models. These are kept separate from the database models on purpose - schemas validate and document the public API surface,

while ORM models represent the actual stored data; keeping them separate means internal fields (like `password_hash`) can never accidentally leak into an API response. Notable details: `IngredientUpdate.quantity` uses `Field(gt=0)` so the API rejects a non-positive quantity outright (the frontend calls DELETE instead of PATCH-to-zero); the `*Out` schemas set `from_attributes=True` so Pydantic can build the response directly from a SQLAlchemy object; and `RegenerateRecipeRequest.reason` captures which of the four predefined dislike reasons was picked (or "other" with free-text details).

1.3 Authentication

backend/app/auth.py

Role: Password hashing, JWT issuing/verification, the auth dependency

Three responsibilities live here. First, password hashing via the `bcrypt` library directly (`hash_password/verify_password`) - `bcrypt` automatically salts each hash, so two users with the same password get different stored hashes. Second, `create_access_token` builds a JWT that encodes the user's ID as the sub claim and an expiry timestamp, signed with the app's secret key. Third, and most used throughout the app, `get_current_user` is a FastAPI dependency that every protected endpoint depends on: it reads the `Authorization: Bearer <token>` header, decodes and verifies the JWT's signature and expiry, looks up the matching User row, and either returns it or raises a 401. Every router in the app (except signup/login) uses `Depends(get_current_user)`, which is how the API enforces that users can only ever see or modify their own data.

1.4 Application Entry Point

backend/app/main.py

Role: FastAPI application entry point

This is where the FastAPI app object is actually created and wired together. It calls `Base.metadata.create_all(bind=engine)` on startup, which creates the SQLite tables if they don't already exist (a no-op on every subsequent run) - a lightweight alternative to a full migration system, appropriate for this project's scope. It configures CORS so the React dev server (running on a different port, 5173) is allowed to call the API on port 8000, and registers all six routers (auth, account, preferences, ingredients, classify, recipes). A `/health` endpoint is included for quick manual checks that the server is up.

1.5 API Routers

Six routers, each owning one resource; every endpoint except `signup/login` requires a valid login token.

backend/app/routers/auth_router.py

Role: Signup and login endpoints

Exposes `POST /auth/signup` and `POST /auth/login`. Signup checks that the username and email aren't already taken, hashes the password, creates the User row, and immediately returns a login token plus `has_preferences: false` - the frontend uses that flag to route brand-new users straight to the onboarding questionnaire rather than the home page. Login verifies the password against the stored hash and returns a token with `has_preferences` set based on whether a `UserPreference` row already exists, so returning users skip the questionnaire.

backend/app/routers/account_router.py

Role: Settings-page profile actions

Two endpoints behind login: `PUT /account` lets a user change their username and/or password (checking the new username isn't already taken by someone else), and `DELETE /account` permanently deletes the account. Because every related table was declared with cascading deletes in `models.py`, a single `db.delete(user)` call is enough to clean up all of that user's preferences, ingredients, and recipes.

backend/app/routers/preferences_router.py

Role: Dietary preferences questionnaire, get and set

`GET /preferences` returns the current preferences (404 if none exist yet). `PUT /preferences` does double duty: on a brand-new account it creates the first `UserPreference` row (from the onboarding questionnaire), and on later calls it overwrites the existing row's fields (from Settings) - the same endpoint handles both cases by checking whether `user.preferences` already exists.

backend/app/routers/ingredients_router.py

Role: CRUD for "what's in my fridge"

`GET /ingredients` lists everything the user currently has. `POST /ingredients` adds a new one - with a small but important detail: if an ingredient with the same name (case-insensitive, via `.ilike()`) already exists for that user, its quantity is increased instead of creating a duplicate row, so adding "Tomato" twice results in one row with quantity 2, not two separate rows. `PATCH /ingredients/{id}` updates a quantity (rejecting non-positive values via the schema), and `DELETE /ingredients/{id}` removes one. A shared `_get_owned_ingredient` helper enforces that a user can only ever fetch/modify their own ingredients, returning 404 otherwise.

backend/app/routers/classify_router.py

Role: Photo → ingredient name

A single `POST /classify` endpoint that accepts an uploaded image file. Before doing anything else it calls `classifier.is_model_available()`, and if the trained model files aren't present on disk yet, it returns a clean 503 with an explanatory message instead of crashing - a deliberate design choice so the rest of the app keeps working even before the ML model was trained and deployed. Otherwise it reads the uploaded bytes, hands them to `classifier.predict()`, and returns the predicted ingredient name plus a confidence score.

backend/app/routers/recipes_router.py

Role: Recipe generation, favorites, and history

The most involved router. Its core is a private `_generate_and_save` helper: it loads the user's current ingredients, calls `openai_service.generate_recipe(...)` to get a recipe back from OpenAI, and saves

the result as a new Recipe row - which is also how the “every recipe automatically lands in History” requirement works, since History is just every row for that user. POST /recipes/generate is the entry point from the home screen's meal-type buttons. POST /recipes/{id}/regenerate is the “thumbs down” flow: it looks up the disliked recipe, converts the chosen reason (or free-text “other”) into a readable note, and - critically - passes the original recipe's full content back into _generate_and_save so the AI prompt can be told exactly what to avoid repeating. PATCH /recipes/{id}/favorite toggles the heart icon. GET /recipes/favorites and GET /recipes/history power the two list pages (filtered vs. unfiltered by is_favorite), and GET /recipes/{id} fetches one recipe's full detail.

1.6 Machine Learning — Ingredient Classifier

backend/app/ml/classifier.py

Role: Loads the trained model and runs inference

Wraps the trained ingredient-recognition model (a MobileNetV2-based Keras model, produced by the training scripts described in section 1.8) for use inside the FastAPI app. The model and its class-name list are loaded lazily - only on the first actual classification request, and cached in module-level globals afterward - so the API can start up instantly without waiting for TensorFlow to load a large model file. `predict()` opens the uploaded image with Pillow, converts it to RGB, resizes it to 224×224 (matching exactly what the model was trained on), adds a batch dimension, and runs it through the model to get a probability distribution over all ingredient classes; it returns the single highest-probability class name and its confidence score. Two subtle but important fixes live in `_load_model()`: `safe_mode=False` is required because the saved model contains a Lambda layer (MobileNetV2's own image preprocessing step) that Keras blocks from loading by default as a security precaution, and `builtins.keras = keras` works around a real bug where that same Lambda layer's serialized code can't find the `keras` name in its execution scope when reloaded in a different process - without this line, every classification request crashes with a `NameError` even though the model loads successfully.

1.7 AI Recipe Generation

backend/app/services/openai_service.py

Role: Builds the recipe prompt and calls OpenAI

This is where “what should the AI actually be asked” is decided. A fixed `SYSTEM_PROMPT` instructs the model to act as a home-cooking assistant, explicitly tells it that it doesn't need to use every ingredient the user has (only the handful - roughly 4 to 8 - that actually belong in the dish), to strictly respect allergies and disliked foods, and to respond with nothing but a JSON object in an exact shape (title, prep_time, difficulty, ingredients_used, steps). `_build_user_prompt` assembles the per-request context: the meal type, the user's full ingredient inventory with quantities, and a human-readable summary of their dietary preferences. When regenerating after a dislike, it appends the previously-shown recipe's title and ingredient list plus the user's stated reason, and explicitly instructs the model to produce something clearly different - not just a small variation - since each API call is otherwise stateless and the model would have no way to know what it had already suggested. `generate_recipe()` sends this as a two-message chat completion (system + user) with `response_format={"type": "json_object"}` to guarantee valid JSON back, and raises a clear `RuntimeError` up front if no API key is configured, which the router turns into a graceful 503 rather than a crash.

1.8 Model Training Scripts

Standalone scripts (not part of the running app) that produce the model files consumed by classifier.py. See section 3 for the results of running both.

backend/training/train_model.py

Role: First model — from-scratch CNN

Reproduces the training approach from the original project notebook so it can be run locally instead of in Colab. It merges the two source datasets (a fruit/vegetable image set and the Freiburg groceries dataset) into one folder by category, loads them at 128×128 resolution, and builds a CNN completely from scratch: three convolution + max-pooling blocks (32, 64, and 128 filters), followed by a dense layer, dropout for regularization, and a softmax output over all ingredient categories. It trains for 10 epochs, evaluates on a

held-out test split, and saves the resulting model and class-name list straight into backend/app/model/. This was the first model actually deployed into the app, before its per-category weaknesses led to a second attempt.

backend/training/train_model_mobilenet.py

Role: Second model — MobileNetV2 transfer learning

Mirrors a later, improved version of the training notebook. Instead of learning visual features from scratch, it loads MobileNetV2 pretrained on ImageNet with its classification head removed and its weights frozen, and builds a new model on top: random flip/rotation/zoom layers for data augmentation, MobileNetV2's own preprocessing step, the frozen pretrained base, global average pooling, dropout, and a new dense softmax layer sized to the app's ingredient categories. It reuses the dataset already merged by the first script rather than redoing that step, trains at a larger 224×224 resolution for 10 epochs, and overwrites the deployed model files. This is the model actually running in the app today.

2. Frontend

The frontend lives under `frontend/src/`. It's a React app built with Vite, styled mobile-first (a phone-width column centered on wider screens) with plain CSS rather than a UI framework. Pages own their data and talk directly to the backend through a shared API client; reusable pieces of UI live in a separate components folder.

2.1 App Shell & Routing

frontend/src/main.jsx

Role: App bootstrap

The actual entry point Vite loads. It renders the React tree into the page's root DOM node, wrapping the whole app in `BrowserRouter` (enables client-side page navigation) and `AuthProvider` (makes login state available everywhere), and imports the global stylesheets (theme, components, and page-specific CSS) once at the top level.

frontend/src/App.jsx

Role: Route definitions and access control

Defines every page route with React Router and decides which ones require login. A small `RequireAuth` wrapper component redirects to `/login` if the user isn't authenticated; every route except `/login` and `/signup` is wrapped in it. A separate `AppLayout` wrapper adds the bottom navigation bar around the six main pages (but not Login, Signup, or the Questionnaire, which don't show it). Any unmatched URL falls back to the home route.

2.2 API Communication & Auth State

frontend/src/api/client.js

Role: Thin fetch wrapper for talking to the backend

A small helper so every page doesn't have to repeat the same fetch boilerplate. It builds the full request URL, automatically attaches the saved JWT as an `Authorization` header when one exists, JSON-encodes the request body (or passes a `FormData` object through untouched for the photo-upload endpoint), and - importantly - turns any non-2xx response into a thrown `JavaScript Error` with the backend's actual error message, so every page can just wrap an API call in `try/catch` and display `err.message` directly to the user.

frontend/src/context/AuthContext.jsx

Role: Global login state

A React Context that holds the current JWT and username, backed by `localStorage` so a page refresh doesn't log the user out. It exposes `login`, `signup`, `logout`, and `updateUsername` functions to the rest of the app via the `useAuth()` hook. `updateUsername` exists specifically to fix a bug found during testing: without it, renaming your account in Settings updated the database but not the "Hi, {username}" greeting on the home page, since nothing told the in-memory auth state to refresh.

2.3 Pages

Each page owns its own state and talks to the backend directly through the api client - there's no separate global state-management library, since the app's data needs are simple enough for React's built-in hooks.

frontend/src/pages/LoginPage.jsx & SignupPage.jsx

Role: Authentication forms

Simple controlled forms bound to component state. On success, login checks the returned `has_preferences` flag to decide whether to go straight to the home page or first to the questionnaire; signup always goes to the questionnaire, since a new account never has preferences yet.

frontend/src/pages/QuestionnairePage.jsx

Role: Onboarding preferences questionnaire

Shown once, immediately after signup. Renders the shared `PreferencesForm` component with empty defaults and, on submit, PUTs the answers to `/preferences` before navigating to the home page.

frontend/src/pages/HomePage.jsx

Role: Main screen

Loads the user's ingredients on mount for the horizontal "fridge" scroller, renders the six meal-type buttons, and drives the whole recipe-generation flow: tapping a button opens a confirmation modal, confirming calls `/recipes/generate`, and the result is shown in the full-screen `RecipeOverlay`.

frontend/src/pages/IngredientsPage.jsx

Role: Ingredient inventory management

Lists every ingredient with +/- quantity controls and a delete button. Decrementing below 1 calls the delete endpoint directly rather than sending an invalid non-positive quantity to the API.

frontend/src/pages/RecipesPage.jsx & HistoryPage.jsx

Role: Favorites and history lists

Nearly identical list pages backed by different endpoints (`/recipes/favorites` vs. `/recipes/history`); tapping any item opens it in the same `RecipeOverlay` used everywhere else, so favoriting or regenerating from History behaves exactly the same as from Home.

frontend/src/pages/SettingsPage.jsx

Role: Account and preferences management

Combines account editing (username/password), preference editing (reusing `PreferencesForm` again, this time pre-filled), logout, and account deletion behind a confirmation step.

2.4 Reusable Components

Pieces of UI shared across multiple pages, kept in their own folder so pages themselves stay focused on layout and data-fetching.

frontend/src/components/BottomNav.jsx

Role: Bottom tab bar

The five-tab bar shown on every logged-in page except the questionnaire. The center “+” button doesn't navigate anywhere - it opens `AddIngredientModal` directly, and on a successful add, redirects to the Ingredients page so the user sees the result.

frontend/src/components/HomeButton.jsx

Role: Back-to-home link

A small house-icon link back to /, added to the top of every non-Home page since Home itself isn't one of the five bottom-nav tabs and there would otherwise be no way back to it.

frontend/src/components/FridgeScroller.jsx

Role: Horizontal ingredient chip list

The horizontal, side-scrolling ingredient chip list on the home screen; purely presentational, driven entirely by props.

frontend/src/components/AddIngredientModal.jsx

Role: Add-ingredient popup

A two-step popup. Step one offers manual entry or a photo. Choosing a photo immediately uploads it to /classify and, on success, pre-fills the name field with the model's prediction before moving to a confirmation step where the user can still edit the name, quantity, and unit before saving.

frontend/src/components/MealTypeConfirmModal.jsx

Role: Meal-type confirmation popup

A simple yes/no confirmation before generating a recipe; for the “Other” meal type it first asks the user to type what they want.

frontend/src/components/RecipeOverlay.jsx

Role: Full-screen recipe view

The most visually developed component in the app. Displays the recipe in cards (ingredients, steps) with a decorative header banner and badges for prep time/difficulty, and owns the favorite-toggle and thumbs-down-feedback logic, calling back up to the parent page via `onRecipeUpdated` whenever the recipe changes (favorited, or replaced entirely by a regenerated one).

frontend/src/components/ThumbsDownFeedbackModal.jsx

Role: Dislike-reason popup

Presents the four predefined dislike reasons (too long, too difficult, not to my taste, other) as selectable options, with a free-text field that appears only for “other”, and hands the choice back to `RecipeOverlay` to trigger regeneration.

frontend/src/components/PreferencesForm.jsx

Role: Shared dietary-preferences form

Used by both the onboarding questionnaire and Settings. Internally simplifies the two separate `is_vegetarian/is_vegan` boolean fields into a single three-option dropdown (none/vegetarian/vegan) for a

clearer UI, converting back to the two booleans on submit; comma-separated text fields for allergies and disliked foods are split into arrays before being sent to the API.

3. Models & Results Summary

Two ingredient-recognition models were trained during this project, both on the same merged dataset (fresh fruits/vegetables + Freiburg groceries, ~26,800 images across 61 categories).

	Model 1 - From-scratch CNN	Model 2 - MobileNetV2 (deployed)
Approach	3 conv/pool blocks trained from scratch, 128x128 images, no augmentation	Pretrained ImageNet backbone (frozen) + augmentation, 224x224 images
Test accuracy	88.8%	94.55%
Fresh produce (e.g. Apple, Tomato)	97-100% precision/recall	97-100% precision/recall
Packaged goods (e.g. Chips, Coffee, Beans)	As low as 8-30% precision	Improved to roughly 50-90% precision
Outcome	Working, but a clear systematic weakness	Deployed - fixes the weakness

The first model reached almost 89% overall test accuracy, but a per-category breakdown revealed it had essentially learned fresh produce very well while failing to generalize to photos of packaged groceries - likely because those categories have far more real-world visual variety (branding, packaging, angles) and fewer training images per class. Rebuilding the model with transfer learning (MobileNetV2 pretrained on ImageNet) and data augmentation closed most of that gap: overall accuracy rose to 94.55%, and the weakest packaged-goods categories improved by several times their original precision. The MobileNetV2 model is the one currently deployed in `backend/app/model/` and used by the running app.

End to end, the final application supports: account creation with an onboarding preferences questionnaire, manual and photo-based ingredient tracking, AI-generated recipes personalized to available ingredients and dietary restrictions, a favorites list, a full recipe history, and a feedback loop where disliking a recipe produces a genuinely different suggestion rather than repeating the same one.